Quarto and publishing

Data Science Flipped Classroom - Module 8

Adriana Clavijo Daza

2026-07-02

Anatomy of a Quarto file

Three kinds of content

A `.qmd` file is plain text, made of just three ingredients:

- A YAML header: the settings for the whole document
- Prose: ordinary text written in Markdown
- Code: chunks of R that run and produce output

> You have used all three since Module 1. Today we look at each one more closely.

How they fit together

````
---
title: "My report"
format: html
---

## Introduction

Some **prose** written in Markdown.

```{r}
1 + 1
```
````

Header at the top, then prose and code chunks in any order.

The YAML header

What is YAML?

YAML is a small language for settings. In Quarto it holds the document metadata: title, author, output format, and more.

It sits at the very top of the file, delimited by three dashes above and below:

```yaml
---
title: "My report"
format: html
---
```

The two rules of syntax

YAML is fussy about two things:

- Write `key: value`, always with a space after the colon
- Show nesting with indentation, using two spaces, never tabs

```yaml
format:
  html:
    toc: true
```

> `format:html` (no space) or a stray tab will break the render. When YAML misbehaves, check colons and indentation first.

Important YAML options

```yaml
---
title: "Penguin report"
subtitle: "A first analysis"
author: "Your Name"
date: "2026-07-02"
format: html
---
```

These set the visible front matter: what the reader sees at the top of the document.

Controlling the output

```yaml
format:
  html:
    toc: true
    number-sections: true
    code-fold: true
    theme: cosmo
---
```

- `toc`: add a table of contents
- `number-sections`: number the headings
- `code-fold`: hide code behind a clickable toggle
- `theme`: pick a visual style

Execution options

You can also set defaults for every code chunk in one place:

```yaml
execute:
  echo: true
  warning: false
  message: false
---
```

- `echo`: show the code, not just its output
- `warning` and `message`: hide R's chatter from the report

> A chunk option set on an individual chunk overrides the document default.

Document metadata

The front matter the reader sees, all set at the top level:

Option	What it sets
`title`	The document title
`subtitle`	A secondary line under the title
`author`	Who wrote it
`date`	The date shown on the document
`abstract`	A short summary of the work

> `date: today` fills in the render date for you, so it is never stale.

Contents and navigation

Option	What it does
`toc`	Add a table of contents
`toc-depth`	Lowest heading level to include, e.g. `2`, `3`
`number-sections`	Number the headings
`anchor-sections`	Show a link anchor when hovering a heading

Style and appearance

Option	What it does
`theme`	A Bootswatch theme: `cosmo`, `darkly`, `solar`
`css`	A CSS or SCSS file to style the document
`highlight-style`	Syntax highlighting theme: `arrow`, `kate`, `zenburn`
`mainfont`, `monofont`	Font family for text and for code

Code display

Option	What it does
`echo`	Show the code, not only its output
`code-fold`	Let readers toggle the code open and closed
`code-tools`	A menu to hide, show, or download the code
`code-overflow`	Wide code: `scroll` or `wrap`

Figures

Option	What it does
`fig-align`	Alignment: `default`, `left`, `right`, `center`
`fig-width`, `fig-height`	Default figure size, in inches
`fig-format`	Figure format: `retina`, `png`, `svg`, `pdf`

The full list of options, by format, lives at quarto.org/docs/reference.

Rendering

Turning source into output

A `.qmd` is just source text. Rendering runs the code and produces the finished document.

- In RStudio: the Render button, or `Ctrl/Cmd + Shift + K`
- From the terminal: `quarto render report.qmd`

> `quarto preview` renders and opens a live preview that refreshes every time you save.

Text formatting

Markdown is the prose layer

Between chunks you write in Markdown: a few plain-text marks that Quarto turns into formatted output.

You write `**bold**`, you get **bold**. The source stays readable.

The everyday marks

You write	You get
`*italic*`	*italic*
`**bold**`	**bold**
`` `code` ``	`code`
`[link](url)`	a hyperlink
`# Heading`	a section heading

> The number of `#` sets the heading level: `#` is a top-level section, `##` a subsection.

Lists

Bullet lists start each line with a dash:

```
- first item
- second item
  - a nested item
```

Numbered lists use `1.`, `2.`, and so on. A blank line is needed to separate a list from the paragraph around it.

Images and equations

Insert an image with an exclamation mark:

```
1 ![A caption](figures/plot.png)
```

Write mathematics in LaTeX, between dollar signs:

```
1 The mean is $\bar{x} = \frac{1}{n}\sum x_i$.
```

renders as: the mean is $\bar{x} = \frac{1}{n}\sum x_i$.

Code inside prose

Weave a computed value straight into a sentence with inline code: a backtick, the letter `r`, then an expression.

```
The dataset has `r nrow(penguins)` rows.
```

On render it becomes: “The dataset has 344 rows.”

> Numbers written this way update themselves on every render, so your prose never falls out of sync with your data.

Callout blocks

Quarto offers built-in callout boxes for notes and comments:

```
::: {.callout-note}
Penguin data is from the `palmerpenguins` package.
:::
```

The type sets the colour and icon:

Note

`.callout-note`: general information

Tip

`.callout-tip`: a helpful suggestion

Warning

`.callout-warning`: something to watch out for

Figures and tables

Figures from code

Chunk options, the lines starting with `#|`, control how a plot appears:

````
```{r}
#| label: fig-mass
#| fig-cap: "Body mass by species"
#| out-width: "80%"

ggplot(penguins, aes(species, body_mass_g)) +
  geom_boxplot()
```
````

`fig-cap` adds a caption, `out-width` sets the display size.

Captions and cross-references

Give a chunk a `label` starting with `fig-`, then refer to it by that label anywhere in the text:

```
Species differ clearly in @fig-mass.
```

Quarto numbers the figure and inserts a link, exactly like the cross-references covered earlier in the course.

> Tables work the same way: a `tbl-` label with `#| tbl-cap:`, plus a table function such as `knitr::kable()` that you have already used, gives a captioned, referenceable table.

Citations and references

Three words that get confused

- A citation: a pointer to a source in your text. Whenever you quote, paraphrase, or summarize someone else's idea, an in-text citation follows
- A reference: the detailed description of that source, with author, title, year, publisher, DOI
- The bibliography: the list of references at the end, usually alphabetical by author

> You cite a source in the prose, its reference holds the details, and the bibliography gathers them all into one list.

Where references are stored

The reference details live in a separate file, usually BibTeX (`.bib`). Each entry has a citation key:

```
@book{wickham2023,
  title     = {R for Data Science},
  author    = {Wickham, Hadley and others},
  year      = {2023},
  publisher = {O'Reilly}
}
```

Here the key is `wickham2023`. That is how you refer to this entry.

How a citation flows

You point Quarto at the file in the YAML:

```yaml
bibliography: references.bib
```

- You write `[@wickham2023]` in the text
- Quarto inserts the citation marker there
- It automatically builds the matching reference at the end, i.e., only items that are cited appear in the reference list.

To list works you did not cite or the entire `.bib` file, use the `nocite:` option in the YAML:

```yaml
nocite: |
  @wickham2023, @tidy2014
```

```yaml
nocite: |
  @*
```

Scholarly articles

Quarto for academic writing

Quarto can render a full scholarly article: title, authors, affiliations, abstract, and a formatted reference list, all from one `.qmd`.

The structure is driven entirely by the YAML header.

The article header

```yaml
---
title: "Penguins of the Palmer Archipelago"
author:
  - name: "Jane Doe"
    affiliation: "University of Somewhere"
    email: "jane.doe@example.edu"
abstract: |
  We examine body size across three penguin
  species and find clear differences by island.
bibliography: references.bib
csl: apa.csl
format: pdf
---
```

Reading the header

- `author`, `affiliation`, `email`: who wrote it and where
- `abstract`: a short summary, indented under the `|` block
- `bibliography`: the `.bib` file of references
- `csl`: the citation style that sets the formatting

> The `|` after `abstract:` lets you write a multi-line value that keeps its line breaks.

> Rendering to PDF needs a LaTeX engine. Make sure it's installed before rendering.

Adding citations

Cite at the end of a sentence with the key in square brackets:

```
Tidy data has one row per observation [@wickham2014].
```

renders as: Tidy data has one row per observation (Wickham 2014).

- `[@key]`: a parenthetical citation, in brackets
- `@key`: an in-text citation, “Wickham (2014) showed…”
- `[@key1; @key2]`: cite several works at once

Citations as numbers

By default, citations show author and year. To show them as numbers instead, choose a numeric CSL style:

```
csl: ieee.csl
---
```

Now `[@wickham2014]` renders as `[1]`, and the references are listed in citation order.

> The CSL file alone controls the look. You change the style by swapping the file, never by editing your citations.

Publishing to GitHub Pages

What is GitHub Pages?

GitHub Pages is a free service that serves a website straight from a GitHub repository. It hosts static pages: plain HTML, exactly what Quarto renders.

> Render your `.qmd` to HTML, push it to GitHub, and it is live on the web.

Two requirements

- The repository must be public (on a free account)
- The home page must be named `index.html`

GitHub looks for `index.html` as the entry point, the same way a website's front door is always `index`.

The usual setup

A common layout renders the site into a `docs/` folder:

```
.
└── docs
    ├── index.qmd
    ├── references.bib
    └── index.html
```

In the repository settings, point Pages at the `docs/` folder on the main branch.

Going live

- Render `index.qmd` so `index.html` appears
- Commit and push to GitHub
- Enable Pages in the repository settings
- Visit `https://organization.github.io/repo`

> After the first push, every later push that changes the HTML updates the live site automatically.

A single shareable file

No public repo? Bundle everything into one self-contained HTML file:

```yaml
format:
  html:
    embed-resources: true
---
```

Images, styles, and scripts are folded into the one `.html`, so you can email it or hand it over and it just works.

Slides with Quarto

reveal.js

Quarto builds slides from the same Markdown you already know. The HTML slide format is reveal.js: set it in the YAML.

```yaml
---
title: "My talk"
format: revealjs
---
```

> These very slides are a `revealjs` Quarto document.

Headings make slides

The structure is simple:

- `#` starts a new section, a divider slide
- `##` starts a new content slide
- Everything under a heading is that slide's content

```
## First slide

Some content here.

## Second slide

More content.
```

Building content gradually

To reveal a slide in steps, put a line containing only `. . .` between the parts. Everything below it stays hidden until the next click.

An incremental list reveals one bullet per click:

```
::: {.incremental}
- appears first
- then this one
- then this one
:::
```

Code and output on slides

Code chunks work exactly as in any Quarto document. You write:

````
```{r}
1 + 1
```
````

and on the rendered slide you get both the code and its result:

```
1 + 1
```

```
[1] 2
```

What else can Quarto build?

One source, many outputs

The same Markdown and code render into far more than a single report:

- Documents: HTML, PDF, Word
- Presentations: reveal.js, PowerPoint, Beamer (PDF)
- Websites: like this course's site
- Books: like R for Data Science
- Dashboards: interactive panels, see the gallery
- Manuscripts: reproducible articles, as HTML or as PDF.

> You can also use custom typst formats as templates: https://quarto.org/docs/output-formats/typst.html

Learn more

https://quarto.org/docs/authoring/markdown-basics

https://quarto.org/docs/authoring/citations

https://quarto.org/docs/publishing/github-pages

https://quarto.org/docs/presentations/revealjs

Questions?